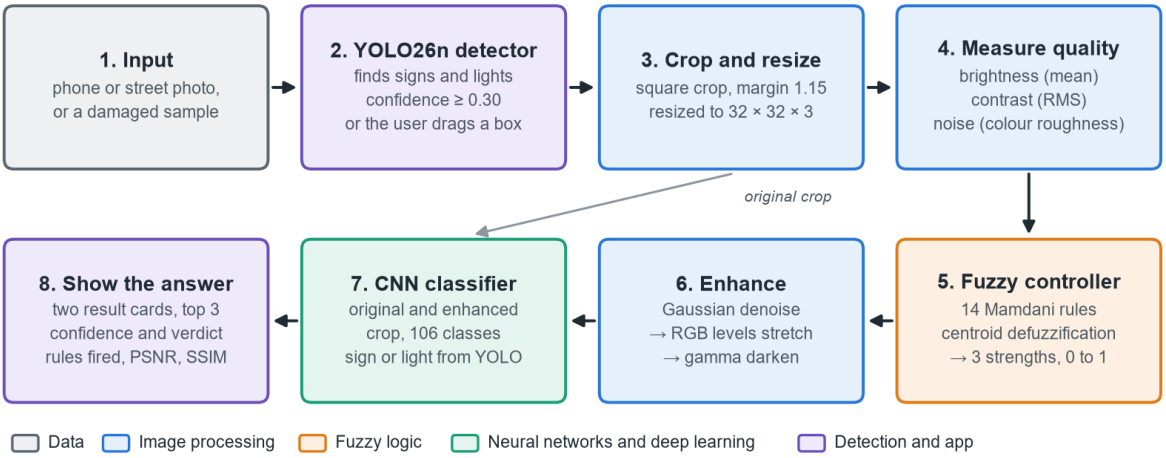
MINI PROJECT REPORT

Traffic Sign and Signal Recognition in Bad Conditions

Fuzzy-logic image enhancement, neural network classification and YOLO detection



One project submitted to three courses

Deep Learning · Image Processing and Analysis · Neural Networks and Fuzzy Logic

Submitted by	Keshav Kumar
Roll number	20241CAI0072
Department	Presidency School of Artificial Intelligence and Advanced Computing (PSAIAC)

November 2026

Abstract

Traffic signs and signals have to be read in conditions that damage the image: darkness, haze, noise, blur, rain, snow and worn paint. This project builds one pipeline for that problem. Each image is measured for brightness, contrast and noise, a Mamdani fuzzy controller decides how strongly to denoise, stretch and darken it, and a neural network names the sign or light from the original and the enhanced image side by side. A YOLO detector finds signs and lights in full photos, and a Streamlit app shows every step.

Seven datasets were used, from 2.2 million CURE-TSR images to Mapillary street photos from six continents and two sets of real damaged signs. On 203,374 real CURE-TSR test images the best 17-class CNN reached **89.8% accuracy and 0.819 macro F1**, and **97.9% accuracy** on 281,547 traffic-light test images. Fuzzy enhancement helped every model trained only on clean images, while training on damaged images helped more. A 106-class CNN that also learned worn paint scored 0.808 on the same real signs and lifted worn real signs from 0.597 to **0.786 macro F1**. Every score is the mean of 3 training runs, chosen on validation data, with each test set scored once.

Table 1. Key results at a glance (test sets, mean of 3 seeds).

Result	Value
Best 17-class CNN, 203,374 real CURE-TSR test signs	89.8% accuracy, 0.819 macro F1
Same CNN on 459,760 simulated signs and 281,547 lights	0.909 macro F1 and 97.9% accuracy
Fuzzy enhancement on the CNN trained on clean images	real signs 0.683 → 0.715 F1, lights 84.7% → 90.9%
106-class CNN with worn signs, 6 datasets	0.808 real signs, 0.959 lights, 0.786 worn signs
Fine-tuned YOLO26n detector, test mAP50	0.701 signs, 0.680 lights, 0.893 damaged close-ups
Automated checks across the code	262 passing

Keywords: traffic sign recognition, bad weather and low light, fuzzy logic, Mamdani inference, image enhancement, CNN, MLP, backpropagation, YOLO, robustness, CURE-TSR, LISA, Mapillary.

Contents

Abstract 2
Contents 3
1. Introduction 4
  1.1 Problem statement 4
  1.2 Objectives 4
  1.3 Coverage of the three courses 4
2. System design 4
  2.1 Workflow 4
  2.2 Architecture 5
  2.3 Technology stack 6
  2.4 Development protocol 6
3. Datasets 6
  3.1 Damage, splits and balance 7
4. Image Processing and Analysis 7
  4.1 Operations implemented 8
  4.2 Removing noise 8
  4.3 Measuring image quality 9
  4.4 Enhancement and image quality 9
  4.5 Simulating worn paint 9
5. Fuzzy Logic Controller 10
  5.1 Design 10
  5.2 Rule base 11
  5.3 Implementation and checks 11
  5.4 Fuzzy enhancement against a fixed setting 11
6. Neural Networks 12
  6.1 Models 12
  6.2 Backpropagation by hand 12
  6.3 MLP settings study 13
  6.4 MLP, feature MLP and CNN compared 13
7. Deep Learning 14
  7.1 CNN architecture and training 14
  7.2 Final 17-class results 15
  7.3 Where the best model goes wrong 16
  7.4 Traffic lights 16
  7.5 Wider models: 84 and 106 classes 16
  7.6 Object detection with YOLO 17
  7.7 Worn and damaged signs 19
8. The Application 20
9. Testing and Validation 22
10. Limitations and Future Scope 22
11. Summary of All Results 23
12. Conclusion 23
References 23
Appendix A. Running the project 24
Appendix B. Key design decisions 24
Appendix C. Metrics used 25

1. Introduction

1.1 Problem statement

Driver assistance has to recognise signs and lights in the conditions where people struggle too: at night, in haze, rain and snow, through a dirty lens, and on old signs with faded paint. A classifier trained on clean images loses most of its value on damaged ones. In this project the same CNN scored 0.683 macro F1 on real damaged test signs when trained on clean images, against 0.819 when trained on damaged ones. The project asks whether classical image processing, steered by fuzzy logic, can recover enough of a damaged image to help a neural network, how that compares with simply training on damaged images, and how far the result carries to real street photos.

1.2 Objectives

1. **Image processing:** build point operations, histogram methods, filters and convolution, and measure their effect with PSNR and SSIM.
2. **Fuzzy logic:** design a Mamdani controller that measures brightness, contrast and noise and chooses how strongly to denoise, stretch and darken.
3. **Neural networks:** train and compare an MLP on pixels, an MLP on HOG and colour features, and an MLP with hand-written backpropagation.
4. **Deep learning:** train CNNs on clean, damaged and enhanced images, scale them to 106 classes across six datasets, add YOLO detection and handle worn signs.
5. **Delivery:** an app that shows each step, and results reported honestly: 3 seeds, choices made on validation, each test set scored once.

1.3 Coverage of the three courses

Table 2. How the project maps onto each course.

Course	Topics covered in this project	Sections
Image Processing and Analysis	negative, gamma, contrast stretch, histogram equalization, CLAHE, convolution, Sobel, Laplacian, noise models, mean, Gaussian, median, bilateral and non-local means filters, PSNR, SSIM, HOG features, synthetic degradation	4
Neural Networks and Fuzzy Logic	fuzzy sets, membership functions, Mamdani inference, centroid defuzzification, MLP, backpropagation by hand, activations, optimizers, learning rates	5, 6
Deep Learning	CNNs, batch normalization, dropout, augmentation, class weighting, training regimes, 106-class transfer to new datasets, YOLO object detection, robustness to wear	7

2. System design

2.1 Workflow

Figure 1 follows one photo through the system. YOLO finds each sign or light, or the user drags a box. The crop is resized to 32 × 32 pixels and measured, the fuzzy controller sets three strengths, and the enhancement runs in a fixed order: denoise first, because stretching would amplify leftover noise, then stretch, then darken. The CNN classifies both the original and the enhanced crop, so the app can show whether enhancement changed the answer.

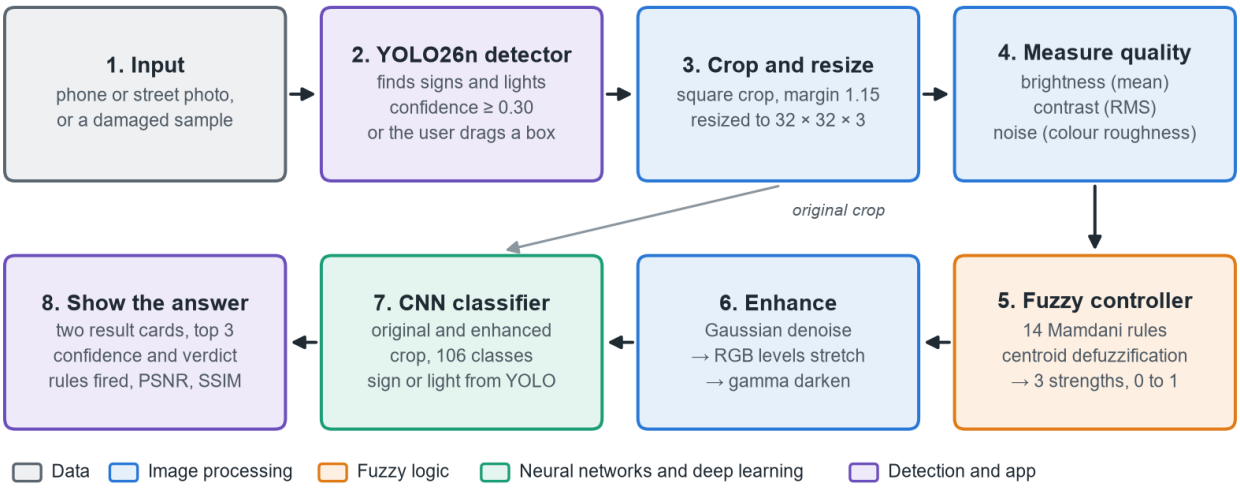


Figure 1. End-to-end workflow for one image. Colours show which course each step belongs to.

2.2 Architecture

The code is split into four packages with no circular links: image_processing, fuzzy_logic and deep_learning hold the methods, experiments runs training and scoring, and app.py is the Streamlit interface. Datasets are packed once into 32 × 32 NumPy arrays, which made training on 1.9 million images practical on a laptop GPU.



Figure 2. System architecture, from data sources to evaluation and the app.

2.3 Technology stack

Table 3. Technology stack.

Layer	Tools and versions
Language and environment	Python 3.11 in a project virtual environment, Windows 11
Deep learning	PyTorch 2.15 with CUDA 13.4 for training, and PyTorch 2.5.1 with CUDA 12.4 from 11 November, on an NVIDIA RTX 4050 laptop GPU (6 GB)
Object detection	Ultralytics 8.4.198, YOLO26 nano pretrained on COCO
Image processing	OpenCV, scikit-image, NumPy, Pillow
Fuzzy logic	scikit-fuzzy membership functions, inference written in NumPy
Evaluation and charts	scikit-learn metrics, pandas, matplotlib
Application	Streamlit with streamlit-cropper; a Cloudflare quick tunnel for phone tests

2.4 Development protocol

The models grew in four stages (Figure 3). At every stage the rule was the same: train 3 seeds, compare on validation data, record the choice in a decision log, then score the test sets once and change nothing afterwards. This keeps every reported test score free of tuning on the test data.

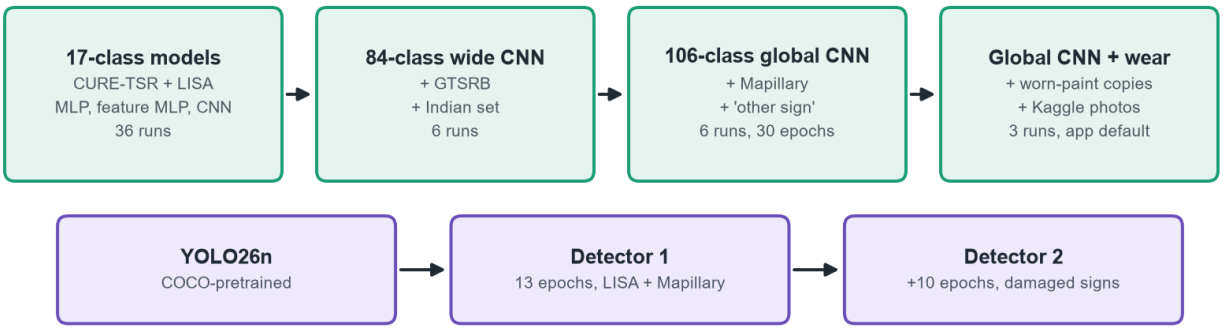


Figure 3. How the classifiers and the detector grew, and the evaluation rule used at every stage.

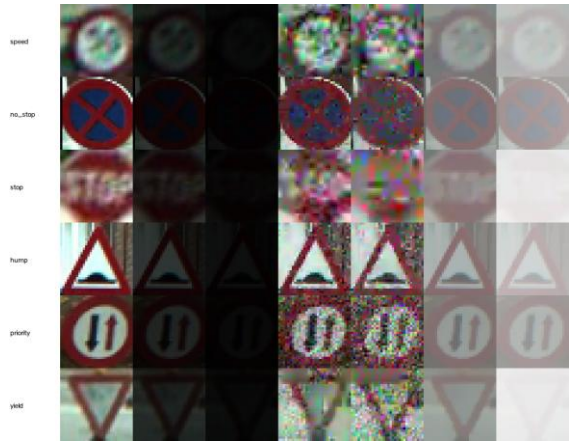
3. Datasets

Seven datasets were combined. CURE-TSR is the core: every damaged image has a matching clean original, so image quality can be measured exactly. The others widen the classes, add real photos and add real damage.

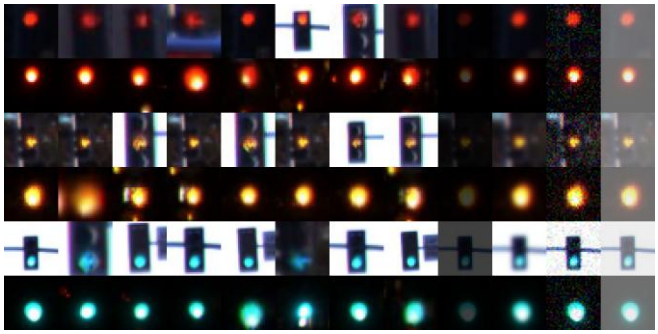
Table 4. Datasets, what each adds and how many images were used. Crops are 32 × 32 pixels.

Dataset	What it adds	Images used (train / val / test)	Classes
CURE-TSR, real and simulated	14 sign types, 12 damage conditions at 5 levels, clean originals	1,542,966 training files; 203,374 real and 459,760 simulated test images	14
LISA Traffic Light	red, yellow and green lights, day and night, split by whole clips	41,923 light crops, plus damaged copies (281,547 test images)	3
GTSRB	real German sign photos; whole tracks held out	33,329 / 5,880 / 12,630 clean crops	43, merged
Indian Traffic Sign Classification	real Indian sign photos; near copies kept out of test	3,307 / 604 / 448 clean crops	67 used
Mapillary Traffic Sign Dataset	street photos from six continents, with an 'other sign' class	36,240 / 6,427 / 6,196 crops; 30,397 detection frames	+22
Kaggle Damaged Signs (CC0)	faded, peeled, rusted and vandalised Lisbon signs	994 / 230 / 230 crops; 2,175 / 460 / 480 frames	3 named
Roboflow Detection_of_damage_signs (CC BY 4.0)	close-ups labelled by kind of damage, for YOLO	1,397 / 399 / 200 frames	boxes only

Class lists grew from **17** (14 CURE-TSR signs and 3 lights) to **84** with GTSRB and the Indian set, and to **106** with Mapillary. The same sign from different datasets was merged into one class, such as GTSRB, Indian and CURE-TSR stop signs.



CURE-TSR: clean, then darkening, noise and haze at levels 3 and 5



LISA: red, yellow and green, day and night, then 4 damages

Figure 4. Samples from the two core datasets, at the 32 × 32 size the models see.

3.1 Damage, splits and balance

- **Damage:** CURE-TSR supplies 12 conditions (decolorization, lens blur, codec error, darkening, dirty lens, exposure, Gaussian blur, noise, rain, shadow, snow, haze) at 5 levels. LISA, GTSRB, the Indian set and Mapillary get darkening, blur, noise and haze added in code: 4 random damaged copies per training image, and all 20 for validation and test. A 13th condition, worn paint, was added later (Section 4.5).
- **No leakage:** neighbouring video frames are nearly identical, so validation holds out the last 15% of each sign's frames with a 5% gap, LISA splits by whole clips, GTSRB by whole tracks, and 768 Indian test images that matched training images by an 8 × 8 average hash were removed. The 11 street photos used to try the app turned out to be Kaggle photos, so they and their neighbours were kept out of every split.
- **Balance:** clean real training signs range from 2,480 parking signs to 38 no stopping signs, so the loss is weighted by inverse class frequency and every score is macro F1.



Kaggle: Lisbon Street View frames, 9 sign groups



Roboflow: close-ups boxed by kind of damage

Figure 5. Labelled frames from the two damaged-sign datasets, as YOLO sees them.

4. Image Processing and Analysis

Course highlights: Image Processing and Analysis

- Point operations, histogram methods, 5 restoration filters and a hand-written 2-D convolution that equals OpenCV on six kernels.
- A noise study on 1,000 real signs: the median filter lifts 2% salt-and-pepper noise from 22.16 dB to 34.83 dB.
- A new colour-roughness noise measure, because the standard estimate barely saw CURE-TSR noise.
- Fuzzy-chosen enhancement raises PSNR over all 13 conditions from 17.90 dB to 18.35 dB while leaving clean images almost untouched.

4.1 Operations implemented

Table 5. Image processing operations built for the project.

Category	Operations	Used for
Point operations	negative, gamma correction, percentile contrast stretch, levels stretch on all RGB channels together	enhancement, image tools tab
Histogram methods	histogram equalization and CLAHE on lightness only, so colours do not shift	image tools tab, comparison
Convolution	hand-written 2-D convolution, equal to OpenCV on six kernels; Gaussian, sharpen, Laplacian, Sobel edges	image tools tab, teaching
Noise and restoration	Gaussian and salt-and-pepper noise; mean, Gaussian, median, bilateral and non-local means filters	noise study, denoise step
Quality measures	MSE, PSNR (capped at 60 dB), SSIM; mean brightness, RMS contrast, colour-roughness noise	evaluation, fuzzy inputs
Features	HOG (9 orientations, 8 × 8 cells, 2 × 2 blocks: 324 values) and 3 colour histograms of 8 bins: 348 values	feature MLP
Degradation	darkening, Gaussian blur, noise, haze and worn paint at 5 levels	training data, robustness tests

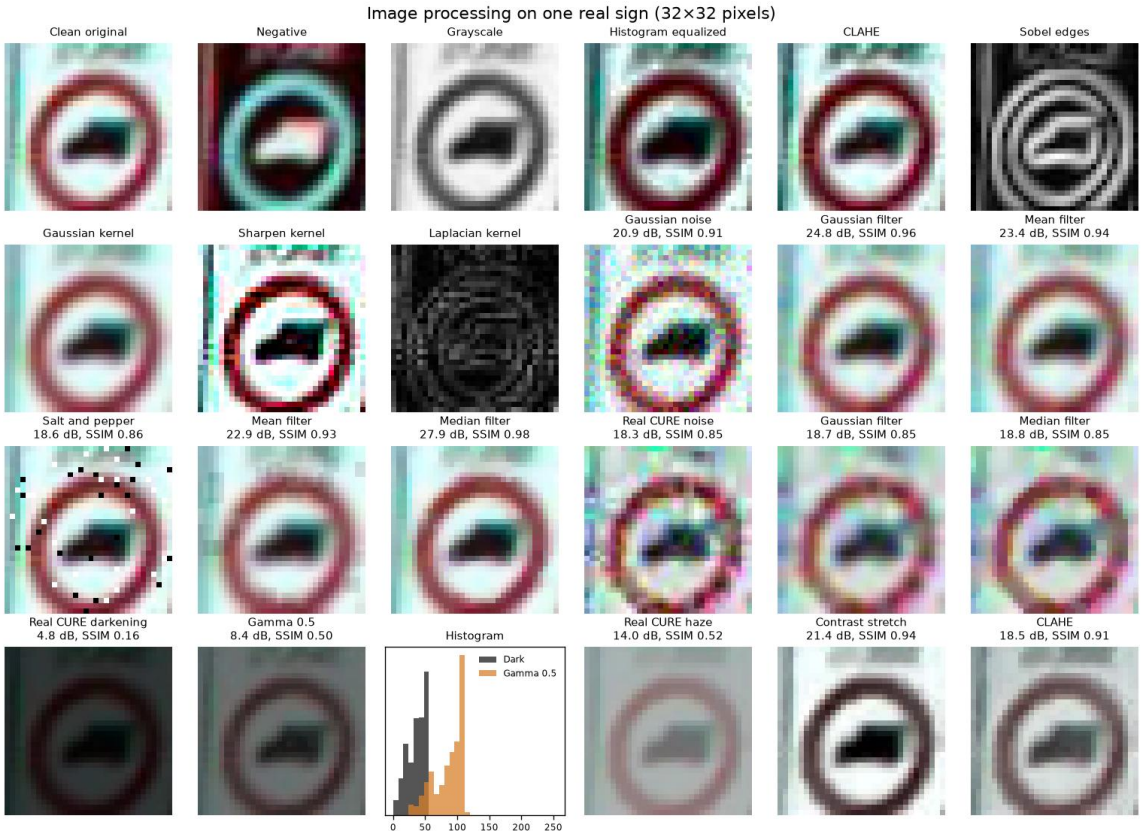


Figure 6. Operations on one real 32 × 32 sign, with PSNR and SSIM against the clean original where damage was added.

4.2 Removing noise

Noise was added to 1,000 clean real test images and each filter's PSNR was measured against the original. The median filter clearly won on salt-and-pepper noise, smoothing filters won on strong Gaussian noise, and non-local means took about 1.4 ms per image, at least 100 times longer than the others, without ever being best.

Table 6. Best filter for each kind of noise, on 1,000 real test signs.

Noise	PSNR before	Best filter	PSNR after	SSIM after
Gaussian, σ = 10	28.24 dB	Bilateral	33.04 dB	0.965
Gaussian, σ = 25	20.51 dB	Gaussian	26.74 dB	0.866
Gaussian, σ = 50	15.03 dB	Mean	22.34 dB	0.725
Salt and pepper, 2%	22.16 dB	Median	34.83 dB	0.968
Salt and pepper, 5%	18.02 dB	Median	33.58 dB	0.964
Salt and pepper, 10%	14.99 dB	Median	31.48 dB	0.955

4.3 Measuring image quality

The fuzzy controller needs three measurements. Brightness is the mean grey level and contrast is the RMS contrast, both from 0 to 1. For noise, the standard Immerkær estimate read 2.0 on clean signs and only 4.0 on level 3 CURE-TSR noise, because that noise is blocky and strongly coloured. The project's measure instead takes the colour channels of the Lab image at half resolution and applies the same Laplacian-style kernel, which separated noise from every other condition far better on training images.

4.4 Enhancement and image quality

Enhanced real test images were compared with their clean originals, using 100 images per condition and level. Fuzzy enhancement raised average PSNR from 17.90 dB to 18.35 dB. A fixed 50% stretch scored 17.40 dB, mainly because it also changes clean images, which the fuzzy controller leaves almost alone (53.37 dB against 28.53 dB).

Table 7. Image quality against the clean original (PSNR in dB, capped at 60).

Condition	PSNR none	PSNR fixed	PSNR fuzzy	SSIM none	SSIM fixed	SSIM fuzzy
All 13 conditions	17.90	17.40	**18.35**	0.57	0.64	0.64
Clean	60.00	28.53	53.37	1.00	0.97	0.99
Darkening	10.44	16.81	**18.16**	0.33	0.78	0.79
Haze	13.45	18.77	18.22	0.52	0.81	0.76
Exposure	7.93	8.30	8.77	0.38	0.39	0.41
Noise	20.70	18.93	20.22	0.63	0.61	0.62
Gaussian blur	17.96	16.54	16.02	0.42	0.46	0.43

4.5 Simulating worn paint

A phone test showed that no dataset contained old, worn signs (Section 7.7). A 13th damage type was written for it: inside a disc around the sign's centre, it fades the colour towards grey and peels smooth random patches, showing the pale backing sheet, while white paint peels to darker metal and black lettering stays. Levels 3 to 5 add rust spots. Traffic lights never get it, because their colour is their label.



Worn paint at levels 1 to 5



Real damaged crops from Kaggle's Lisbon photos

Figure 7. Simulated wear beside real worn signs.

5. Fuzzy Logic Controller

Course highlights: Fuzzy Logic

- Three inputs (brightness, contrast, noise) and three outputs (denoise, stretch, darken), each with three fuzzy sets.
- 14 Mamdani rules with min for AND, max aggregation and centroid defuzzification, written in NumPy for whole batches.
- Matches scikit-fuzzy's engine within 0.0001, and shows which rules fired for every image in the app.
- Lifts the clean-trained CNN on real signs from 0.683 to 0.715 macro F1, and lights from 84.7% to 90.9% accuracy.

5.1 Design

Dark, hazy, noisy and overexposed signs need different fixes, so the controller has three separate outputs instead of one strength. It is a Mamdani system: inputs are fuzzified, 14 IF–THEN rules fire with AND as the minimum, each rule clips its output set, the clipped sets are combined with the maximum, and the centroid gives a strength from 0 to 1.



Written in NumPy for whole batches; within 0.0001 of scikit-fuzzy's engine on 100 random inputs.

Figure 8. The fuzzy controller, from measurements to enhancement.

Table 8. Linguistic variables.

Variable	Role	Range	Fuzzy sets
Brightness	input	0 to 1	dark, normal, bright
Contrast	input	0 to 1	low, medium, high
Noise	input	0 to 60	low, medium, high
Denoise, stretch, darken	outputs	0 to 1	none, weak, strong

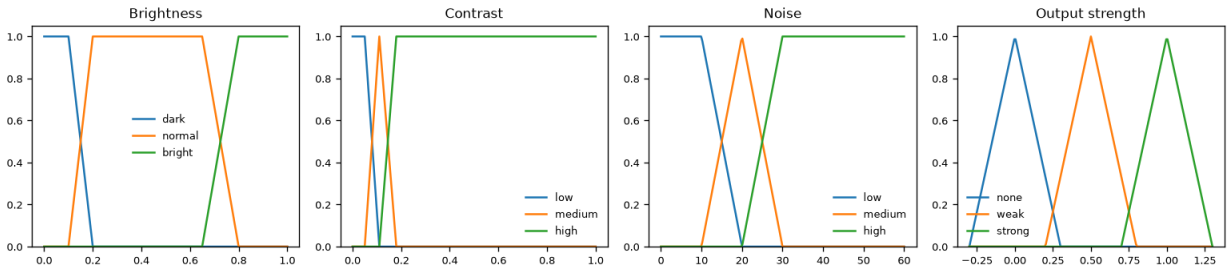


Figure 9. Membership functions for the three inputs and the shared output strength.

5.2 Rule base

Table 9. The 14 rules. Overexposure still has contrast, so bright and flat images are treated as haze and are not darkened.

#	IF	THEN	#	IF	THEN
1	brightness is dark	stretch strong	8	brightness bright AND contrast high	darken strong
2	contrast is low	stretch strong	9	brightness bright AND contrast low (haze)	darken none
3	brightness normal AND contrast medium	stretch weak	10	brightness is normal	darken none
4	brightness normal AND contrast high	stretch none	11	brightness is dark	darken none
5	brightness bright AND contrast medium	stretch none	12	noise is low	denoise none
6	brightness bright AND contrast high	stretch none	13	noise is medium	denoise weak
7	brightness bright AND contrast medium	darken strong	14	noise is high	denoise strong

5.3 Implementation and checks

- Inference is written in NumPy and runs on whole batches, so every packed image could be enhanced in advance on all CPU cores; it also reports which rules fired, which the app displays.
- Its outputs stay within 0.0001 of scikit-fuzzy's own engine on 100 random inputs, one of 36 automated checks of the controller.
- The ranges were tuned on training images only. The first rules fired too often on clean signs (46.92 dB after enhancement); retuning the noise and contrast ranges raised clean signs to 55.27 dB.

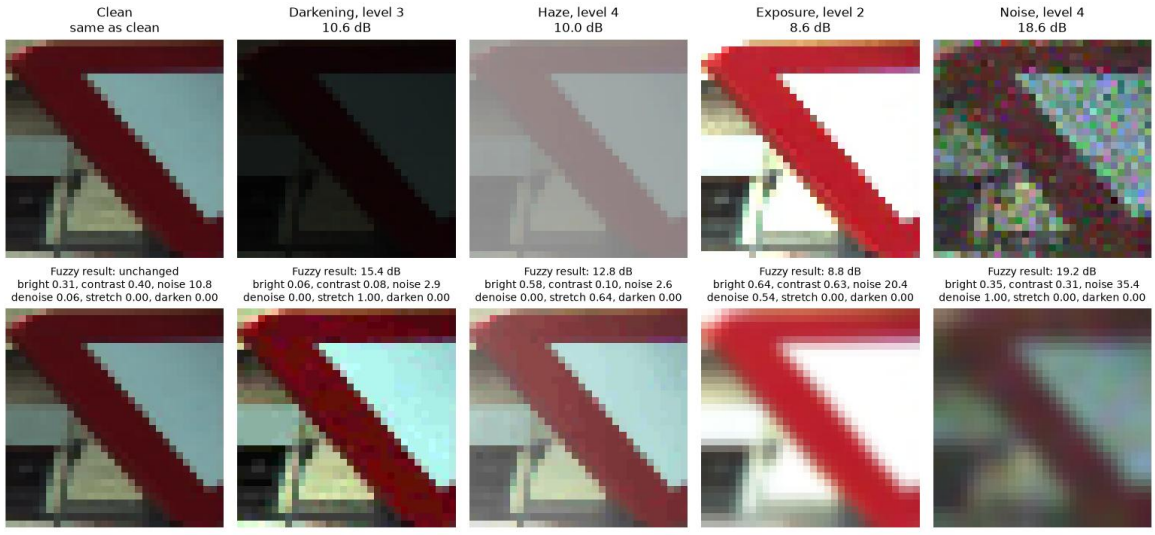


Figure 10. The controller on one real sign under four conditions: measurements, chosen strengths and PSNR before and after.

5.4 Fuzzy enhancement against a fixed setting

To check that fuzzy logic earns its place, 12 fixed settings were tried on validation images with the CNN trained on clean images; the best was a 50% stretch on every image (0.900 validation macro F1, against 0.878 with no enhancement). On the test sets:

Table 10. Fuzzy enhancement against the best fixed setting (test sets, 3 seeds).

Input to the clean-trained CNN	Real signs F1	Simulated F1	Lights accuracy	PSNR, all conditions
Original image	0.683	0.757	84.7%	17.90 dB
Fixed 50% stretch	0.712	0.793	**93.7%**	17.40 dB
Fuzzy enhancement	**0.715**	0.791	90.9%	**18.35 dB**

Fuzzy enhancement and the fixed stretch tie on signs, and the fixed stretch wins on lights, especially in haze (85.0% against 73.0%; 52.9% with no enhancement). The fuzzy rules were tuned for image quality, not for classification, and their clearest advantage is leaving clean images almost untouched. Figure 11 shows where the gains come from: darkening rises from 0.603 to 0.855 and haze from 0.647 to 0.840, clean images stay at 0.868, and blur, codec errors and noise, which these steps cannot undo, stay level or drop by up to 0.02.

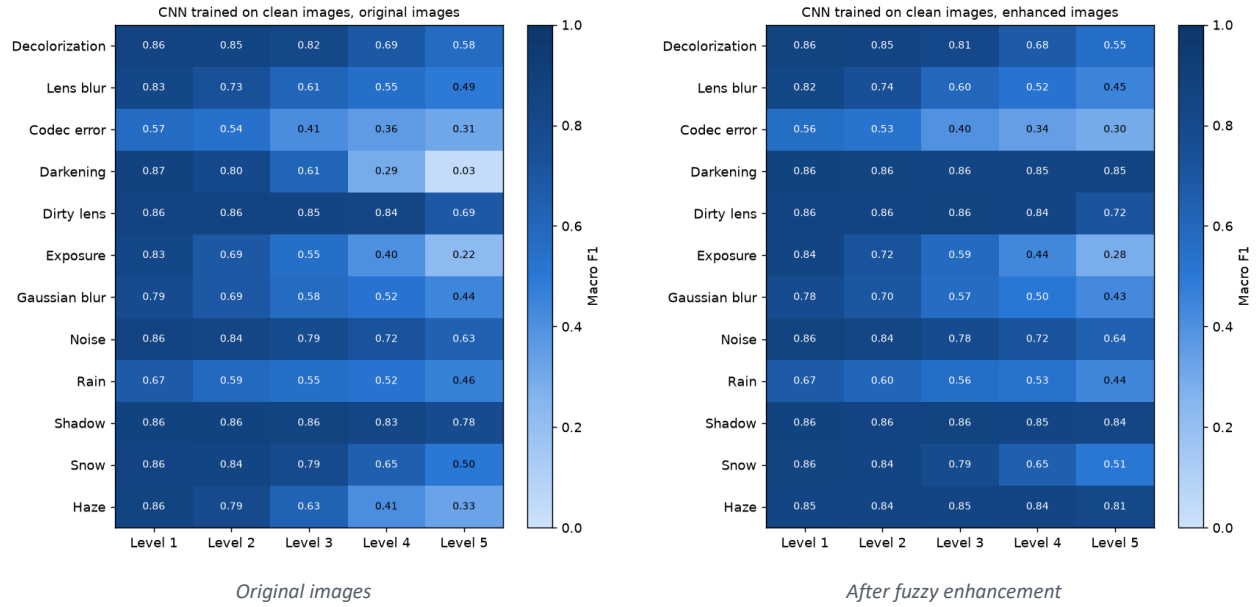


Figure 11. Macro F1 of the clean-trained CNN on real test signs by condition and level, before and after fuzzy enhancement.

6. Neural Networks

Course highlights: Neural Networks

- An MLP on raw pixels, an MLP on 348 HOG and colour features, and an MLP with backpropagation written by hand in NumPy.
- Hand-written gradients pass a finite-difference check with a worst relative error of 5.23×10^{-7}, and match PyTorch step for step.
- A one-at-a-time study of hidden sizes, learning rates, optimizers and activations.
- Raw pixels beat hand-made features: 0.733 against 0.701 macro F1 on real signs when trained on all conditions.

6.1 Models

Table 11. The neural networks compared.

Model	Input	Layers	Weights (17 classes)
MLP	3,072 raw pixels	512 → 256, ReLU, dropout 0.3	1,709,073
Feature MLP	348 HOG and colour values	512 → 256, ReLU, dropout 0.3	314,385
NumPy MLP	1,024 grey pixels	64 hidden, ReLU, written by hand	66,510 (14 classes)
Small CNN	3 × 32 × 32 image	3 convolution blocks, FC 256 (Section 7.1)	816,369

6.2 Backpropagation by hand

PyTorch hides backpropagation, so a small MLP was written in NumPy with every gradient derived by hand. For a batch with inputs X, hidden layer $h = \mathrm{ReLU}(XW_1 + b_1)$ and softmax output p:

- output error $\delta_2 = (p - y) / n$, so $\partial L/\partial W_2 = h^T\delta_2$ and $\partial L/\partial b_2 = \Sigma\, \delta_2$;
- hidden error $\delta_1 = (\delta_2 W_2^T) \odot 1[XW_1 + b_1 > 0]$, so $\partial L/\partial W_1 = X^T\delta_1$;
- update $W \leftarrow W - \eta\, \partial L/\partial W$ with $\eta = 0.05$, batches of 64, for 30 epochs.

A finite-difference check found a worst relative gradient error of **5.23×10^{-7}**, and one training step matched PyTorch's to rounding. Trained on clean signs, it reached about 0.80 validation accuracy, curve for curve with an identical PyTorch model (Figure 12).

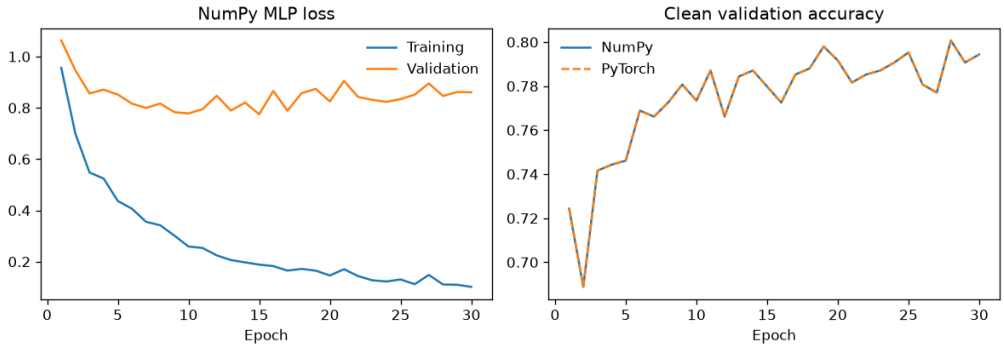


Figure 12. The NumPy MLP: loss, and validation accuracy against the same network in PyTorch.

6.3 MLP settings study

One setting was changed at a time, training on clean images for 30 epochs. A learning rate of 0.01 collapsed to 0.526, and the largest network scored lowest of the sizes. Each setting ran once, so small gaps may be chance.

Table 12. MLP settings study (validation macro F1, one run each).

Setting changed	Value	Best val. F1	Setting changed	Value	Best val. F1
Baseline	512-256, Adam, 0.001, ReLU	0.728	Learning rate	0.0001	0.745
Hidden sizes	128	0.747	Learning rate	0.01	0.526
Hidden sizes	256-128	0.738	Optimizer	SGD, 0.01	0.740
Hidden sizes	512-256-128	0.732	Optimizer	SGD, 0.1	0.720
Hidden sizes	1024-512	0.715	Activation	sigmoid / tanh	0.745 / 0.736

6.4 MLP, feature MLP and CNN compared

Table 13. The three architectures on the 17-class test sets (original input, 3 seeds).

Model	Trained on	Real signs F1	Simulated F1	Lights F1
CNN	all conditions	**0.819**	**0.909**	0.980
MLP	all conditions	0.733	0.878	0.960
Feature MLP	all conditions	0.701	0.825	0.898
CNN	clean images	0.683	0.757	0.894
MLP	clean images	0.630	0.741	0.878
Feature MLP	clean images	0.591	0.671	0.838

The CNN wins every comparison. The MLP on raw pixels beat the MLP on HOG and colour features in both regimes, so hand-made features lost more than they gave at 32 × 32 pixels. Training on damaged images lifted every model more than any architecture change.

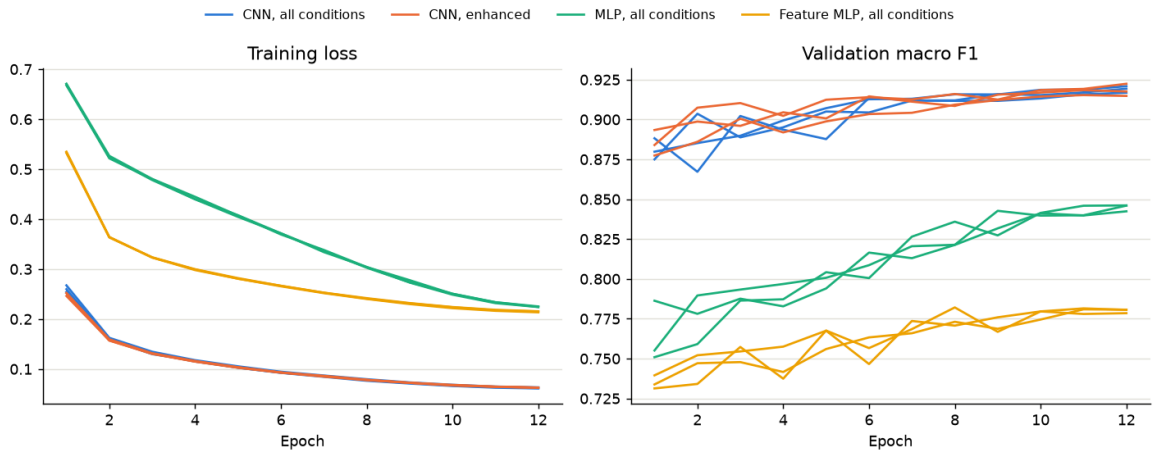


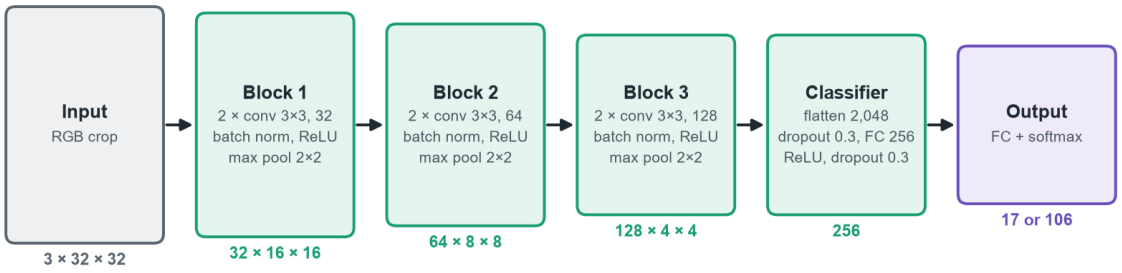
Figure 13. Training loss and validation macro F1 for the models trained on all conditions, 3 seeds each.

7. Deep Learning

Course highlights: Deep Learning

- A CNN with six convolution layers, batch normalization and dropout; 51 classifier runs across 4 class lists, plus 2 YOLO detectors.
- Training on damaged images beats enhancing them: 0.819 against 0.715 macro F1 on 203,374 real damaged test signs.
- Scaled from 17 to 106 classes over six datasets without losing accuracy on the original signs (0.816 against 0.819).
- Worn-sign training raised worn real signs from 0.597 to 0.786, and fine-tuning YOLO raised damaged close-ups from 0.003 to 0.893 mAP50.

7.1 CNN architecture and training



816,369 weights for 17 classes and 839,242 for 106. AdamW, learning rate 0.001, cosine schedule, batch 256, class-weighted loss.

Figure 14. The CNN used for every classifier, from 17 to 106 classes.

Table 14. Training setup.

Setting	Value
Optimizer and schedule	AdamW, learning rate 0.001, weight decay 0.0001, cosine annealing
Batch and loss	256 images; cross-entropy weighted by inverse class frequency
Augmentation	small random rotation, shift and zoom on the GPU; never flips, which can change a sign's meaning
Regimes	clean images only; all conditions; and fuzzy-enhanced copies of either
Epochs and stopping	17 classes: 60 clean or 12 all-conditions epochs, patience 12 or 4 on validation macro F1. 106 classes: 30 epochs, patience 10
Seeds and hardware	3 seeds per model; RTX 4050, about 154 to 171 s per epoch for the 106-class CNN on 1.7 to 1.9 million images

Table 15. Every training run in the project: 51 classifier runs for the results, 12 for the studies, and 2 detectors.

Model family	Data	Classes	Runs	Epochs	Training / validation images
MLP, feature MLP, CNN	CURE-TSR + LISA; clean, all conditions, enhanced	17	36	60 or 12	CURE-TSR's 1.54 million training files, less the validation blocks, plus LISA
Wide CNN	+ GTSRB, Indian set	84	6	60 or 12	clean or all-conditions sets of the four datasets
Global CNN	+ Mapillary	106	6	60 or 30	1,725,061 / 406,372 (all conditions)
Global CNN + wear	+ worn-paint copies, Kaggle crops	106	3	30	1,918,825 / 477,955; 93 minutes per run
MLP settings study	CURE-TSR, clean	14	11	30	one run per setting
NumPy MLP	CURE-TSR, clean, grey	14	1	30	compared with PyTorch
YOLO detector 1	LISA + Mapillary frames	2	1	13	35,729 frames; 5.0 hours
YOLO detector 2	+ Kaggle and Roboflow frames	2	1	+10	39,301 frames; 4.4 hours

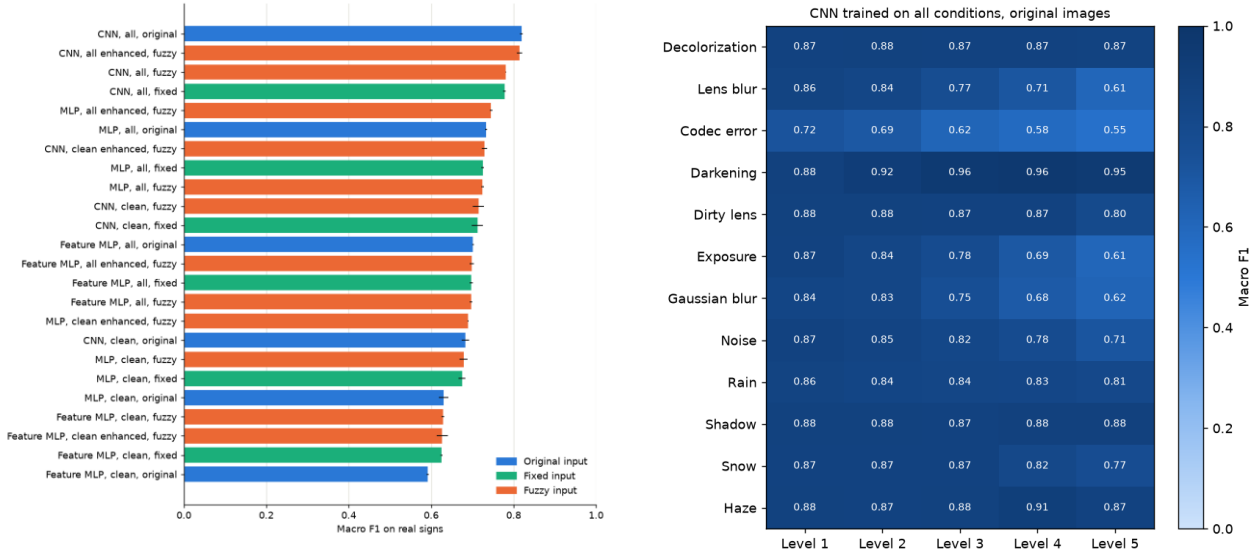
7.2 Final 17-class results

All 24 combinations of model, training regime and input were scored on 944,681 test images: 203,374 real signs, 459,760 simulated signs and 281,547 lights. The main ones:

Table 16. 17-class test results, macro F1 as mean ± spread of 3 seeds.

Model	Trained on	Input	Real signs F1	Simulated F1	Lights F1
CNN	all conditions	original	**0.819 ± 0.003**	**0.909 ± 0.001**	0.980 ± 0.002
CNN	enhanced images	enhanced	0.814 ± 0.006	0.901 ± 0.001	**0.981 ± 0.003**
CNN	all conditions	enhanced	0.781 ± 0.001	0.875 ± 0.002	0.979 ± 0.003
MLP	enhanced images	enhanced	0.745 ± 0.003	0.886 ± 0.002	0.956 ± 0.015
MLP	all conditions	original	0.733 ± 0.003	0.878 ± 0.001	0.960 ± 0.003
CNN	clean images	enhanced	0.715 ± 0.014	0.791 ± 0.004	0.936 ± 0.010
Feature MLP	all conditions	original	0.701 ± 0.002	0.825 ± 0.001	0.898 ± 0.003
CNN	clean images	original	0.683 ± 0.009	0.757 ± 0.008	0.894 ± 0.011

- **Training on damage beats enhancing:** the CNN rose from 0.683 to 0.819 on real signs when trained on all conditions; fuzzy enhancement alone took it to 0.715.
- **Enhancement helps models that never saw damage,** and slightly hurts a model that did (0.819 → 0.781), because it changes what that model expects. Training on enhanced images removes most of that cost (0.814).
- **Validation was the easier check:** the best CNN scored 0.902 on validation, whose frames come from the training videos, and 0.819 on the real test set.



All 24 combinations, real signs *Best CNN by condition*

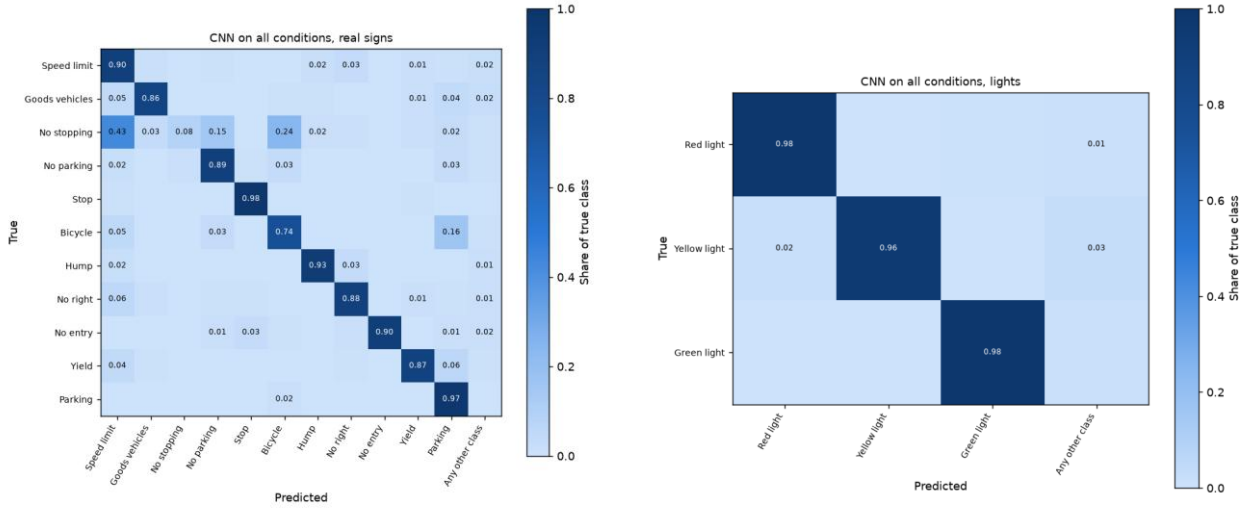
Figure 15. Macro F1 of every combination, and of the best CNN for each of the 13 conditions, on real test signs.

7.3 Where the best model goes wrong

Most of the drop from validation to test comes from one sign. No stopping had only 38 clean training images; on the test set just 8% of its 976 images were read correctly and 43% were read as speed limit. Its F1 of 0.114 pulls the macro average down, while the other ten real signs average 0.889. The same sign explains why darkening (0.941) scores above clean images (0.876) in Figure 15.

Table 17. Per-class results of the best CNN on the real test set.

Class	F1	Recall	Test images	Class	F1	Recall	Test images
Green light	0.989	0.984	126,000	No right	0.896	0.880	19,154
Red light	0.987	0.981	126,000	Hump	0.894	0.928	5,612
Yellow light	0.963	0.955	29,547	Speed limit	0.884	0.896	32,818
Parking	0.946	0.970	72,895	Goods vehicles	0.840	0.855	3,965
Stop	0.911	0.981	2,684	Bicycle	0.807	0.742	23,546
No entry	0.911	0.904	2,623	**No stopping**	**0.114**	**0.083**	976
No parking	0.906	0.889	22,692	Yield	0.900	0.871	16,409



Real test signs *Test lights*

Figure 16. Confusion matrices of the best CNN, as shares of each true class.

7.4 Traffic lights

Lights held up on LISA's four unused test sequences, falling from 99.9% on validation to 97.9%. The best CNN read lights about equally well by day (97.4%) and at night (98.3%); yellow was the weakest colour at 95.5% recall, and about 1.2% of lights were mistaken for signs.

Table 18. Signs and lights side by side for the CNN (real signs, test).

CNN trained on	Input	Sign accuracy	Sign macro F1	Light accuracy
All conditions	original	89.8%	0.819	97.9%
Enhanced images	enhanced	89.6%	0.814	97.9%
All conditions	enhanced	87.5%	0.781	97.4%
Clean images	enhanced	79.9%	0.715	90.9%
Clean images	fixed stretch	79.8%	0.712	93.7%
Clean images	original	75.3%	0.683	84.7%

7.5 Wider models: 84 and 106 classes

Seventeen classes proved too few for street photos, because any other sign got a wrong name. A wide CNN added GTSRB and the Indian set (84 classes), and a global CNN added Mapillary's street signs from six continents and an 'other sign' class (106 classes). The global CNN needed 30 epochs: at 12 it scored only 0.69 on Mapillary validation signs.

Table 19. Macro F1 of the wide and global CNNs on each test set (original input unless stated, 3 seeds).

Test data	Wide CNN, all cond.	Global CNN, all cond.	Global CNN, clean	Global clean + fuzzy	17-class CNN
Real CURE-TSR signs	0.757	**0.816**	0.713	0.745	0.819
Simulated signs	0.881	**0.888**	0.769	0.800	0.909
Traffic lights	0.946	**0.951**	0.839	0.941	0.980
GTSRB	0.907	**0.940**	0.821	0.893	–
Indian set	0.613	0.719	**0.742**	**0.742**	–
Mapillary	–	0.797	0.750	**0.815**	–

Adding Mapillary cost nothing on the old datasets: the global CNN beats the wide CNN everywhere and comes within 0.003 of the 17-class CNN on real signs while knowing 89 more classes. Fuzzy enhancement again helped the clean-trained model on five of six datasets, most on lights (0.839 → 0.941).

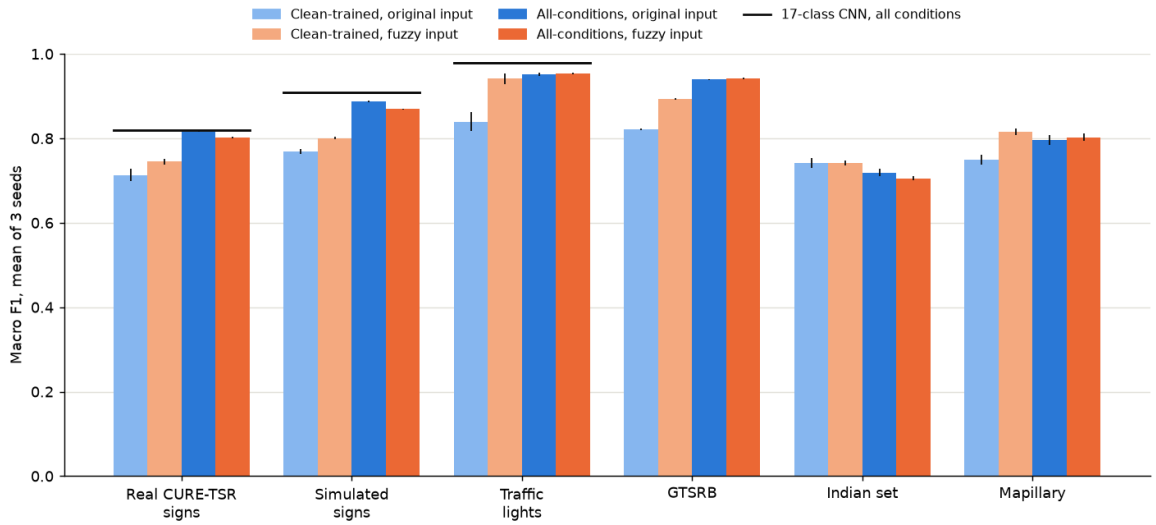


Figure 17. The 106-class global CNN on every test set; black lines mark the 17-class CNN.

7.6 Object detection with YOLO

YOLO26 nano, pretrained on COCO, learned two classes, sign and light, from full frames at 1,280 pixels: LISA frames for lights and Mapillary frames for signs, split exactly as for the classifiers. Mapillary does not label lights, so COCO's model marked lights it was at least 50% sure of in Mapillary's training and validation frames; test frames keep only human labels. It trained for 13 epochs (5.0 hours), and was later fine-tuned for 10 more epochs with the two damaged-sign datasets (4.4 hours). A confidence threshold of 0.30 was chosen from validation F1 curves.

Table 20. YOLO on test frames. A find counts when its box overlaps a labelled box by at least half.

Test data	Frames	Detector 1 precision / recall	Detector 1 mAP50	Detector 2 precision / recall	Detector 2 mAP50
LISA lights	18,252	0.713 / 0.615	0.663	0.786 / 0.597	**0.680**
Mapillary signs	5,210	0.803 / 0.618	0.694	0.805 / 0.625	**0.701**
Kaggle damaged signs	480	0.721 / 0.764	0.749	0.835 / 0.806	**0.853**
Roboflow damaged close-ups	200	0.414 / 0.005	0.003	0.872 / 0.798	**0.893**

Detector 2's F1 on validation peaks near 0.30 confidence for signs (0.29 to 0.36 across the three sign sets) and stays flat up to about 0.49 for lights, so the threshold of 0.30 was kept (Figure 18).

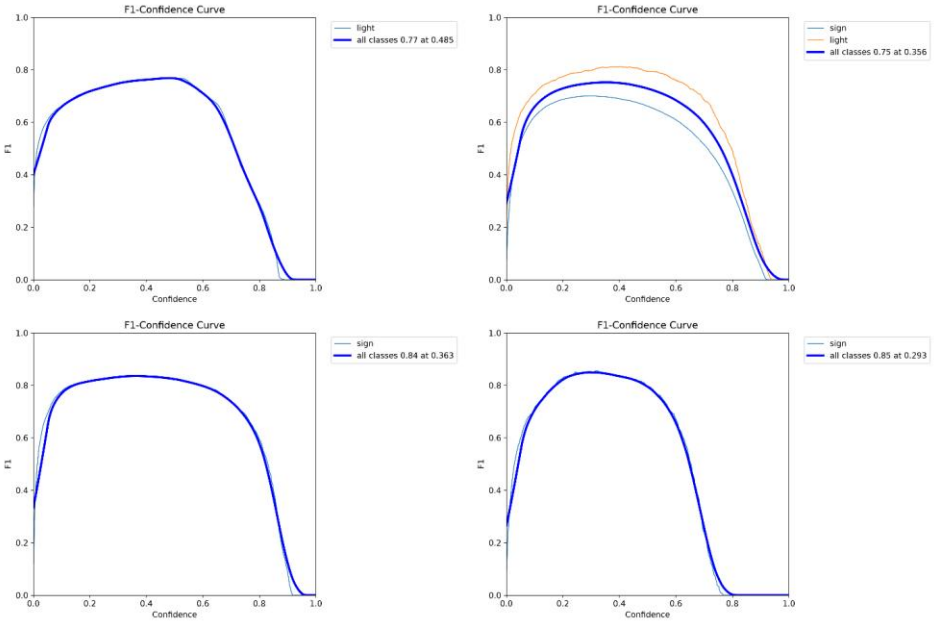
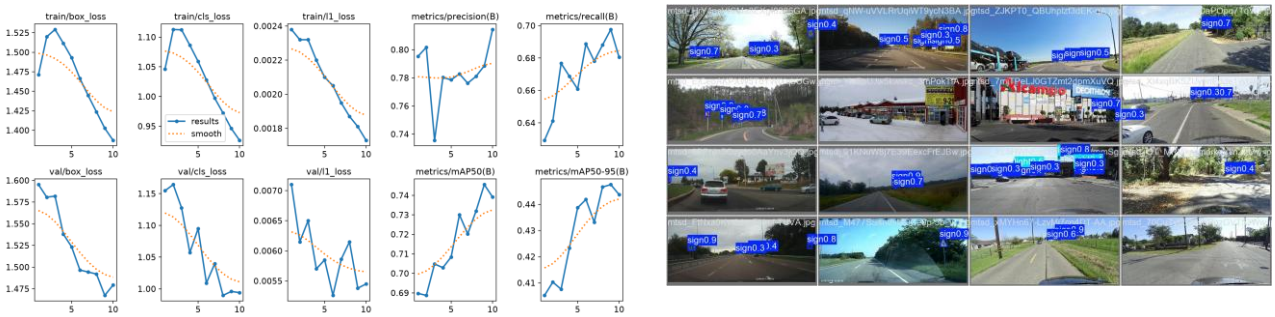


Figure 18. Detector 2's F1 against confidence on the validation frames of LISA, Mapillary, Kaggle and Roboflow.

On 11 street photos, detector 1 found all 18 hand-marked signs and lights. The global CNNs then named two close-up green lights as signs, so the app now lets the classifier choose only among lights when YOLO says light, and only among signs when it says sign; this raised correct names from 12 to 14 of 16 without changing any reported score.



Fine-tuning curves over 10 epochs *Predictions on Mapillary validation frames*

Figure 19. YOLO fine-tuning: losses fall and validation mAP50 rises from 0.690 to 0.746; boxes on unseen street frames from several countries.



Figure 20. YOLO boxes on four of the downloaded street photos: signs in yellow, lights in green.

7.7 Worn and damaged signs

A phone test found a weakness no test set covered: worn paint. On a photo of an old stop sign with most of its red peeled off, every model named another sign and YOLO found nothing. Painting the peeled areas red again made the clean global CNN answer stop at 87%, so the models were relying on colour. A third global CNN was trained with worn-paint copies of every sign image and the Kaggle crops, and YOLO was fine-tuned with both damaged-sign datasets.

Table 21. The global CNN with and without worn signs (test sets, original input, 3 seeds).

Test data (macro F1)	Global CNN, all conditions	Global CNN + worn signs	Change
Worn real CURE-TSR signs	0.597	**0.786**	+0.19
Worn simulated signs	0.678	**0.900**	+0.22
Worn GTSRB	0.629	**0.867**	+0.24
Worn Indian set	0.440	**0.592**	+0.15
Worn Mapillary	0.469	**0.649**	+0.18
Real damaged photos (Kaggle)	0.968	0.968	0
Real CURE-TSR signs	0.816	0.808	−0.008
Traffic lights	0.951	0.959	+0.008

The choice was made on validation before the test sets were touched (Table 22). The worn-sign CNN was far better on worn signs but lower by more than the seed spread on simulated signs, GTSRB and Mapillary, so it missed the stricter rule written before training; it was chosen anyway, openly, because worn signs are common on real roads. Detector 2 was better on every validation set and replaced detector 1 under the rule.

Table 22. Validation scores used for the choice (3 seeds, original input).

Validation set (macro F1)	All conditions	+ worn signs	Validation set (macro F1)	All conditions	+ worn signs
Real CURE-TSR signs	0.833	0.830	Worn real signs	0.633	**0.855**
Simulated signs	0.889	0.881	Worn simulated signs	0.734	**0.911**
Traffic lights	0.996	0.996	Worn GTSRB	0.644	**0.875**
GTSRB	0.943	0.936	Worn Indian set	0.405	**0.540**
Indian set	0.704	0.693	Worn Mapillary	0.464	**0.642**
Mapillary	0.797	0.783	Kaggle damaged photos	0.963	0.962

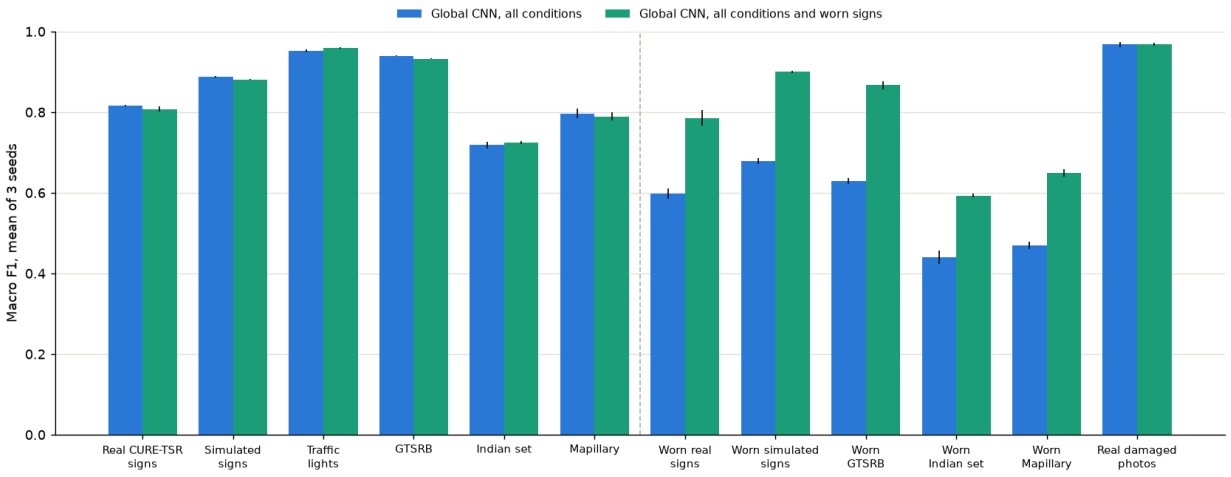


Figure 21. Every test set without (blue) and with (green) worn-sign training; worn and damaged sets are right of the dashed line.

Worn-sign scores rose by 15 to 24 points on the test sets while every usual set moved by under 1 point. The faded stop sign that started this work is now found by YOLO at 73% confidence, but no CNN names it yet: about three quarters of its red is gone and its white letters have turned dark, past the strongest simulated wear. It is kept as a known limitation, rather than tuning the wear to one photo.



The phone-test photo | *What the CNN sees, three crop sizes* | *Peeled areas repainted*

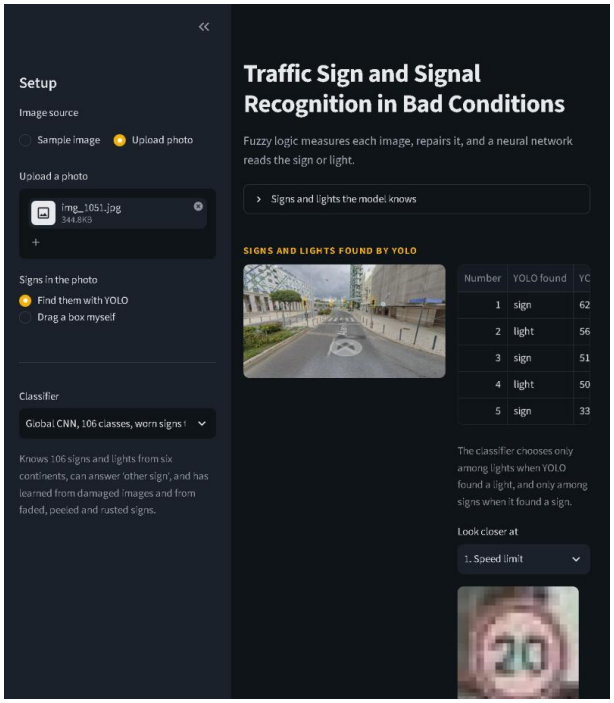
Figure 22. The faded stop sign: every model named it as another sign until the red was painted back, which made the clean global CNN answer stop at 87%.

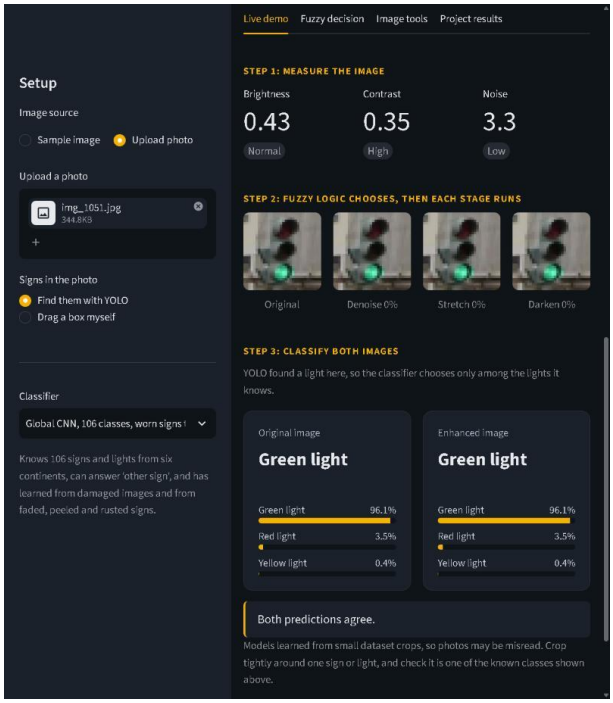
8. The Application

The Streamlit app runs the classifiers on the laptop's processor, and YOLO on the GPU when one is free; it has four tabs. The sidebar picks a sample image (any of the 17 classes, under any condition and level) or an uploaded photo, and one of four classifiers: the worn-sign global CNN, the clean global CNN, and the 17-class CNN and MLP.

Table 23. The app's four tabs.

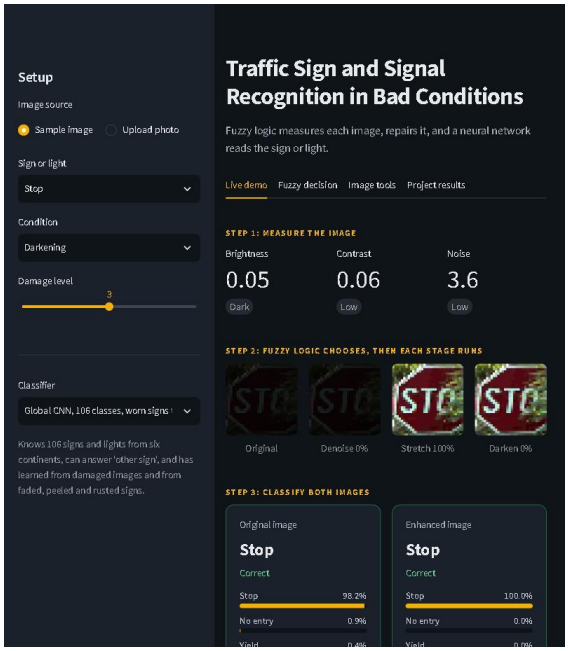
Tab	What it shows
Live demo	YOLO's boxes or a drag box, the three measurements, each enhancement stage, both predictions with their top 3 classes, a verdict, and PSNR and SSIM for samples
Fuzzy decision	where the image falls in each membership function, and every rule that fired with its strength
Image tools	negative, gamma, histogram equalization, CLAHE, kernels, noise and filters on the chosen image
Project results	every test table in this report, including both detectors



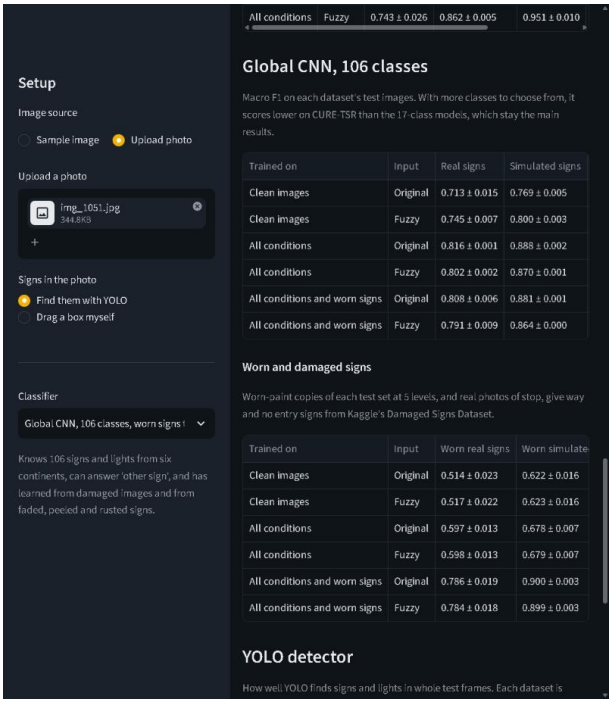


The worn-sign YOLO finds five objects in a street photo | *Fuzzy stages and both answers: green light*

Figure 23. The working app on a downloaded street photo (screenshots taken on 11 November 2026 with the worn-sign CNN and detector).



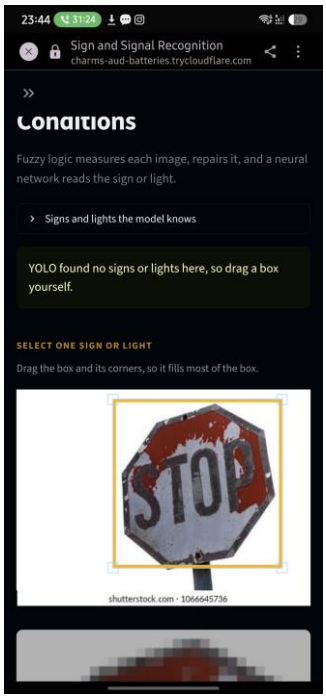
Sample: a darkened stop sign read correctly



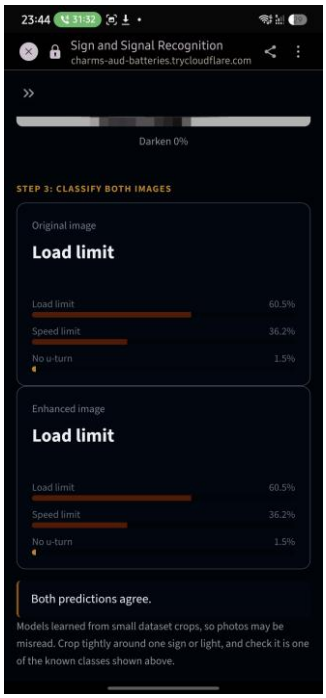
Project results tab

Figure 24. A damaged CURE-TSR sample from the sidebar, and the results tab with every test table.

For phone tests the app was opened through a temporary Cloudflare tunnel. The two screenshots below are from that test: YOLO found no sign in a photo of a faded stop sign, so a box was drawn by hand, and every model answered load limit. That wrong answer is what led to the worn-sign work in Section 7.7.



Drawing a box by hand



Both answers: load limit

Figure 25. The app on a phone, in the test that exposed worn signs.

9. Testing and Validation

Table 24. Automated checks: 262 in total, all passing.

Check suite	Checks	What it proves
Image processing	43	convolution equals OpenCV on six kernels; filters, metrics and transforms behave as defined
Fuzzy logic	36	membership sets, rules and centroid match scikit-fuzzy within 0.0001
Deep learning and detection	108	no image on both sides of a split, models train, YOLO finds at least half the labelled objects on all four test sets, worn paint behaves as designed
App	75	every tab, classifier, condition and class runs headlessly without errors

Table 25. Problems found in real data, and the fixes.

Problem found	What was done
Standard noise estimate read 2.0 on clean signs and only 4.0 on level 3 CURE-TSR noise	measured colour roughness at half resolution instead
Gamma 0.5 reached about 13 dB on level 3 darkening	stretched all RGB channels together, reaching about 21 dB
Fuzzy rules fired too often: clean signs scored 46.92 dB after enhancement	retuned the noise and contrast ranges, raising clean signs to 55.27 dB
Neighbouring frames are nearly identical	held out whole blocks of frames for validation, with a gap before each block
On 11 November, Windows Smart App Control blocked PyTorch 2.15's library files	moved to PyTorch 2.5.1: every saved model loads unchanged, all 262 checks pass, and the app's CNN scores within 0.001 F1 of its stored validation scores
Real test set lacks no overtaking, no left and priority	kept all 14 signs, and reported real and simulated test results separately
Global CNN undertrained, and early stopping cut one run at epoch 14	trained 30 epochs with patience 10, reaching 0.80 on Mapillary validation signs
The 11 test street photos were byte-for-byte Kaggle photos	left them and their neighbours, 79 photos, out of every split

10. Limitations and Future Scope

- **32 × 32 input:** numbers and words are hard to read, so every speed limit is one class and a faded STOP cannot be read from its letters. A larger input or a pretrained backbone could fix this, at the cost of redoing the comparisons.
- **Extreme fading:** wear training fixed most worn signs but not a stop sign that lost three quarters of its red. Stronger sun-bleaching or more real photos of old signs are the next step.
- **Rare classes:** no stopping (38 clean training images) and the Indian set (0.72 macro F1, most classes under 30 photos) need more data.
- **Detection in the dark:** enhancement runs only after YOLO finds a sign; training YOLO on damaged frames, or enhancing the whole photo first, would help night scenes.
- **Real photos in bad conditions:** the phone tests so far were daytime photos; night and rain photos are still to be collected.

11. Summary of All Results

Every headline test score in one place. Each is the mean of 3 seeds on original input, macro F1 unless stated, scored once after the choices were made on validation.

Table 26. All classifier test scores.

Model	Classes	Real signs	Simulated	Lights	GTSRB	Indian	Mapillary	Worn real
Feature MLP, all conditions	17	0.701	0.825	0.898	–	–	–	–
MLP, all conditions	17	0.733	0.878	0.960	–	–	–	–
CNN, clean images + fuzzy	17	0.715	0.791	0.936	–	–	–	–
CNN, all conditions	17	**0.819**	**0.909**	0.980	–	–	–	–
CNN, enhanced images	17	0.814	0.901	**0.981**	–	–	–	–
Wide CNN, all conditions	84	0.757	0.881	0.946	0.907	0.613	–	–
Global CNN, all conditions	106	0.816	0.888	0.951	**0.940**	0.719	0.797	0.597
Global CNN, clean + fuzzy	106	0.745	0.800	0.941	0.893	**0.742**	**0.815**	–
Global CNN + worn signs (app)	106	0.808	0.881	0.959	0.932	0.724	0.789	**0.786**

Table 27. Detection, image quality and other results.

Other result	Value
Best 17-class CNN accuracy	89.8% on 203,374 real signs; 97.9% on 281,547 lights
YOLO detector 2, test mAP50	0.680 LISA lights, 0.701 Mapillary signs, 0.853 Kaggle damaged, 0.893 Roboflow damaged
Fuzzy enhancement, image quality	PSNR 17.90 → 18.35 dB over 13 conditions; clean images kept at 53.37 dB
Best noise filters	median 34.83 dB on 2% salt and pepper; bilateral 33.04 dB on Gaussian σ = 10
NumPy MLP	worst gradient error 5.23×10^{-7}; matches PyTorch step for step
Street photos	18 of 18 signs and lights found; 14 of 16 named correctly with the sign-or-light rule
Real damaged photos (Kaggle crops)	0.968 macro F1 for stop, give way and no entry

12. Conclusion

The project built a complete pipeline for reading traffic signs and lights in bad conditions, covering all three courses in one system: image processing that measures and repairs images, a fuzzy controller that decides how, neural networks from a hand-written MLP to a 106-class CNN, and a YOLO detector that finds signs in full photos. The main lesson is clear and measured: training on damaged images is worth more than repairing them afterwards (0.819 against 0.715 macro F1 on real signs), while fuzzy enhancement still helps any model that has not seen damage and does so without harming clean images. The work also showed the value of testing on real photos: they exposed too few classes, close-up lights and worn paint, and each fix was measured before it was kept.

References

[1] D. Temel, G. Kwon, M. Prabhushankar and G. AlRegib, "CURE-TSR: Challenging Unreal and Real Environments for Traffic Sign Recognition," NeurIPS Workshop on Machine Learning for Intelligent Transportation Systems, 2017.

[2] M. B. Jensen, M. P. Philipsen, A. Møgelmose, T. B. Moeslund and M. M. Trivedi, "Vision for Looking at Traffic Lights: Issues, Survey, and Perspectives," IEEE Transactions on Intelligent Transportation Systems, 2016 (LISA Traffic Light Dataset).

[3] J. Stallkamp, M. Schlipsing, J. Salmen and C. Igel, "Man vs. computer: Benchmarking machine learning algorithms for traffic sign recognition," Neural Networks, 2012 (GTSRB).

[4] C. Ertler et al., "The Mapillary Traffic Sign Dataset for Detection and Classification on a Global Scale," ECCV, 2020.

[5] Indian Traffic Sign Classification dataset, huggingface.co/datasets/kannanwisen/Indian-Traffic-Sign-Classification.

[6] D. Vareta, Damaged Signs Dataset (CC0), kaggle.com/datasets/danielvareta/damaged-signs-dataset; Trafficresearch, Detection_of_damage_signs (CC BY 4.0), universe.roboflow.com.

[7] Ultralytics, YOLO26 models and documentation, docs.ultralytics.com.

[8] E. H. Mamdani and S. Assilian, "An experiment in linguistic synthesis with a fuzzy logic controller," International Journal of Man-Machine Studies, 1975.

[9] J. Immerkær, "Fast noise variance estimation," Computer Vision and Image Understanding, 1996.

[10] Z. Wang, A. C. Bovik, H. R. Sheikh and E. P. Simoncelli, “Image quality assessment: from error visibility to structural similarity,” IEEE Transactions on Image Processing, 2004.

[11] N. Dalal and B. Triggs, “Histograms of oriented gradients for human detection,” CVPR, 2005.

[12] D. E. Rumelhart, G. E. Hinton and R. J. Williams, “Learning representations by back-propagating errors,” Nature, 1986.

Appendix A. Running the project

All commands run from the project folder on Windows. Data preparation reads the original zips without extracting them; every training script skips runs that already finished, so it is safe to stop and restart.

Table 28. Commands to rebuild every result in this report.

Step	Command (inside the project's .venv)
Create the environment	python -m venv .venv, then .venv\Scripts\python.exe -m pip install -r requirements.txt
Check GPU and libraries	python -m scripts.check_setup
Pack CURE-TSR and LISA	python -m scripts.prepare_cure_tsr, then scripts.prepare_lisa, scripts.prepare_features, scripts.prepare_enhanced
Add GTSRB, Indian set, Mapillary	python -m scripts.prepare_wide, scripts.prepare_mapillary, scripts.prepare_global
Add worn and damaged signs	python -m scripts.prepare_wear, scripts.prepare_damaged, scripts.label_lights --sources damaged roboflow
Train the 17-class models	python -m experiments.run_training (36 runs), then experiments.mlp_study and scripts.train_numpy_mlp
Train the bigger CNNs	python -m experiments.run_training --wide, then --mapillary, then --wear
Train YOLO	python -m deep_learning.train_detector, then again with --weights models\detector.pt --epochs 10
Score on validation	python -m experiments.final_scores --split val; experiments.wide_scores --wear --split val; experiments.detector_scores --split val
Score the test sets once	the same three commands with --split test, then experiments.summarize and experiments.wide_charts for the charts
Run all checks	python -m scripts.check_image_processing, scripts.check_fuzzy, scripts.check_deep_learning, scripts.check_app
Start the app	python -m streamlit run app.py, then open http://127.0.0.1:8501

Appendix B. Key design decisions

The project kept a log of 34 decisions with the reason for each and what was rejected. The ones that shaped the results most:

Table 29. Key decisions from the project's decision log.

Decision	Why
Enhance first, then classify, and show both answers	classification gives the MLP, backpropagation, F1 and confusion matrices the courses need, and the side-by-side view shows what enhancement changes
CURE-TSR as the main sign dataset	12 labelled conditions at 5 levels, each with a clean original, so PSNR and SSIM can be measured; GTSRB and others have no condition labels
Three fuzzy outputs instead of one strength	dark, hazy, noisy and overexposed signs need different fixes; denoising goes first because stretching amplifies noise
Hand-written Mamdani inference	runs on whole batches, reports the rules that fired, and was checked against scikit-fuzzy
Train on clean images and on all conditions	the clean model shows what enhancement can do; the all-conditions model shows whether enhancement still adds anything
Weighted loss and macro F1	clean real training signs range from 2,480 parking to 38 no stopping
No flips in augmentation	a flip can change a sign's meaning, such as no left into no right
Choose on validation, test once, keep weak results	dropping no stopping would raise the best score from 0.819 to 0.889, but that would be choosing what to count after seeing test results
Bigger class lists beside, not instead of, the 17-class results	street photos needed more classes, but the finished 17-class experiments stay the graded results
YOLO finds, the CNN names	keeps fuzzy enhancement and the compared classifiers at the centre; YOLO only needs boxes, so any dataset with boxes can teach it
Worn-sign CNN as the app default, against a stricter pre-written rule	15 to 24 points better on worn signs for under 1 point lost elsewhere; the override is recorded openly

Appendix C. Metrics used

TP, FP and FN are true positives, false positives and false negatives for one class; macro averages give every class the same weight however rare it is.

Table 30. Metrics used in this report.

Metric	Definition	Used for
Accuracy	correct predictions ÷ all predictions	overall reading rate
Precision	TP ÷ (TP + FP): how many answers of a class were right	classifiers and YOLO
Recall	TP ÷ (TP + FN): how many images of a class were found	classifiers and YOLO
F1 and macro F1	2 × precision × recall ÷ (precision + recall), averaged over classes	every classifier score
MSE and PSNR	mean squared error; PSNR = $10 \log_{10}(255^2 \div \text{MSE})$ in dB, capped at 60	image quality
SSIM	structural similarity of brightness, contrast and structure, from 0 to 1	image quality
IoU	overlap area ÷ union area of a predicted and a labelled box	matching YOLO boxes
mAP50 and mAP50-95	mean area under the precision–recall curve at IoU 0.5, and averaged over IoU 0.5 to 0.95	YOLO
Mean ± spread	mean and sample standard deviation over 3 training seeds	every reported model score